“MEDICINE INVENTORY MANAGEMENT”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

MVL Samhita	**2520030509**
G Rishitha	**2520030101**
B Vida Swetha	**2520030100**
K Rishitha	**2520030367**

Under the Esteemed Guidance of

DR Y PRASANTHI

Assistant Professor

Department of Computer Science and Engineering

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Academic Year 2026–2027

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

DECLARATION

The Project Report entitled “MEDICINE INVENTORY MANAGEMENT” is a record of bonafide work of MVL Samhita - 2520030509, G Rishitha - 2520030101, B Vida Swetha - 2520030100, and K Rishitha - 2520030367, submitted in partial fulfillment for the award of B.Tech in Computer Science and Engineering (or) Computer Science and Information Technology to K L (Deemed to be) University. The work presented in this report is based on the project files and implementation supplied by the team. The results embodied in this report have not been copied from any other department, university, or institute.

MVL Samhita – 2520030509

G Rishitha – 2520030101

B Vida Swetha – 2520030100

K Rishitha – 2520030367

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the mini project-based report entitled “MEDICINE INVENTORY MANAGEMENT” is a bonafide work done and submitted by MVL Samhita - 2520030509, G Rishitha - 2520030101, B Vida Swetha - 2520030100, and K Rishitha - 2520030367 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of Computer Science Engineering, K L (Deemed to be) University, during the academic year 2026–2027.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the guidance, encouragement, and support of many people. We express our sincere gratitude to everyone who assisted us directly or indirectly during the development of the Medicine Inventory Management system.

We express our greatest appreciation to our Course Coordinator, Dr. Y. Prasanthi, and our project guide, Dr. Yerragudipadu Subbarayudu, for their valuable guidance, suggestions, encouragement, and continuous support throughout the project.

We are grateful to the Head of the Department and the faculty members of the Department of Computer Science Engineering and the Department of Computer Science and Information Technology for providing the facilities and academic environment required to complete this project.

We also thank the teaching and non-teaching staff, our classmates, friends, and family members for their cooperation and support. Their encouragement helped us complete the project and prepare this documentation.

MVL Samhita – 2520030509

G Rishitha – 2520030101

B Vida Swetha – 2520030100

K Rishitha – 2520030367

ABSTRACT

Medicine Inventory Management is a web-based application developed to organize the medicines, batches, suppliers, expiry information, and stock transactions maintained by a pharmacy or healthcare inventory unit. The project addresses the practical difficulty of tracking medicine quantities and expiry dates when inventory information is maintained in registers or disconnected files.

The application provides a medicine catalogue containing medicine name, generic name, category, manufacturer, strength, unit, minimum stock threshold, selling price, and prescription requirement. Batch records maintain batch number, supplier, manufacturing date, expiry date, initial quantity, current quantity, purchase price, and quarantine status. The system calculates medicine-level stock from the associated batches and classifies stock as available, low, or out of stock. It also identifies expired batches and batches approaching expiry.

Supplier management maintains vendor contact details and the relationship between suppliers and supplied batches. The stock transaction ledger records stock-in, stock-out, and adjustment activities with quantity, batch, reason, date, and user details. Business rules prevent stock-out quantities from exceeding available stock and prevent issuing expired stock. The dashboard summarizes inventory indicators and presents stock distribution and recent stock movement information.

The supplied implementation uses React 18, Vite, React Router, custom CSS, and Recharts for the user interface. The project also includes an Express server, MySQL connection through mysql2, REST API route modules, and a MySQL Workbench SQL script. The frontend service layer supports browser local-storage persistence and attempts to communicate with the backend API. The project archive contains a separate buyer-facing medicine browsing and order workflow. Demand forecasting is a stated project objective, but a dedicated forecasting algorithm is not present in the inspected source code; it is therefore documented as a proposed enhancement rather than a completed feature.

The main objective is to improve inventory visibility, support timely expiry monitoring, maintain a traceable record of stock movement, and provide a structured foundation for future demand analysis and replenishment planning.

INDEX

S. No.	Chapter	Topics	Page No.
1	Introduction	1.1 Background; 1.2 Problem Statement; 1.3 Scope; 1.4 Objectives; 1.5 Importance; 1.6 Target Users	7–12
2	System Requirements	2.1 Hardware; 2.2 Software; 2.3 Development Tools	13–15
3	Technology Stack	3.1 Front End; 3.2 Back End; 3.3 Database; 3.4 Version Control; 3.5 APIs and Services	16–20
4	System Architecture	4.1 High-level architecture; 4.2 Layers; 4.3 Deployment architecture	21–22 (left blank)
5	Design	5.1 Functional design; 5.2 ER / Database schema; 5.3 Data validation	23–25
6	Implementation	6.1 Module-wise implementation; 6.2 Front-end logic; 6.3 API integration; 6.4 Authentication	26–32
7	Features	7.1 Main features; 7.2 Feature workflows and business rules	33–36
8	Testing	8.1 API testing; 8.2 Integration testing; 8.3 Tools; 8.4 Test cases	37–40
9	Deployment	9.1 Setup and execution; 9.2 Environment configuration; 9.3 Hosting	41–43
10	Challenges & Limitations	10.1 Issues; 10.2 Solutions; 10.3 Current limitations	44–46
11	Future Enhancements	11.1 Planned features; 11.2 Demand forecasting and integrations	47–49
12	Conclusion	12.1 Summary; 12.2 Achievements; 12.3 Skills learned	50–52
13	References	Books, frameworks, libraries, and documentation	53
14	Appendices	Setup guide, sample API routes, user guide, website screenshots (left blank)	54 onward

CHAPTER - 1
INTRODUCTION

1.1 Background of the Project

Medicines are essential products in hospitals, pharmacies, clinics, and healthcare distribution centres. A pharmacy may handle many medicine names, brands, strengths, pack sizes, suppliers, and batches. Each batch has its own manufacturing date, expiry date, received quantity, purchase cost, and remaining quantity. As the number of medicines and batches increases, tracking this information manually becomes difficult and can lead to stock mismatches, delayed replenishment, and medicines remaining unnoticed after expiry.

Medicine inventory differs from ordinary stock management because expiry and batch traceability are important. Two batches of the same medicine may have different expiry dates and different available quantities. Therefore, the inventory should not be represented only by one total quantity per medicine. A batch-level design makes it possible to identify which lot should be issued first, which lot is close to expiry, and which supplier provided the stock.

The Medicine Inventory Management project provides a centralized digital environment for maintaining the medicine catalogue, supplier directory, batch records, expiry status, and stock movement history. The dashboard presents key inventory indicators, while dedicated pages provide search, filtering, sorting, pagination, and record-management operations. The project archive also contains a buyer-facing area for browsing medicines, managing a cart, placing orders, and viewing order status.

The implementation is built around a React-based user interface and a service layer that loads and updates inventory data. Browser local storage is available as a persistence mechanism, while an Express server exposes REST endpoints and connects to a MySQL database. The supplied SQL script defines the relational database for medicines, batches, suppliers, users, stores, buyer addresses, carts, orders, and related records.

The project is intended to reduce reliance on paper registers and scattered spreadsheets. It provides a clearer view of physical stock, usable stock, low-stock medicines, expiry alerts, and stock transactions. It also creates a structured data foundation from which demand forecasting and replenishment recommendations can be developed.

1.2 Problem Statement

Manual medicine inventory management requires staff to repeatedly check registers, supplier invoices, batch labels, and stock counts. Such a process makes it difficult to know the exact available quantity of a medicine, identify a specific batch, determine which medicines are nearing expiry, and trace the reason for a stock change. When the same medicine arrives from different suppliers or on different dates, a single aggregate quantity hides important batch-level information.

The problem addressed by this project is the absence of a simple, centralized, and traceable platform for maintaining medicine master data and batch-wise stock. The system must keep supplier and batch relationships, calculate stock from batches, distinguish expired from usable quantities, record stock-in and stock-out activity, and highlight low-stock conditions. It should also provide an organized interface for authorized inventory users.

A further requirement is to make stock movement auditable. Stock receipts, issues, and physical-count adjustments should be recorded with the affected medicine and batch, quantity, reason, and transaction time. The system should reject invalid stock-out operations, including quantities above the available balance and attempts to issue expired lots. Demand forecasting is an additional business objective; however, the inspected

project source does not contain a dedicated forecasting model, so forecasting is treated as a planned enhancement.

1.3 Scope of the Project

The functional scope covers medicine master records, batch-level inventory, supplier records, expiry monitoring, stock transactions, dashboard summaries, and a buyer-facing medicine ordering workflow included in the supplied project archive. Medicine records store identifying and descriptive information. Batch records maintain manufacturing and expiry dates, supplier association, initial and current quantities, purchase price, and quarantine state. Supplier records maintain company and contact information.

The inventory dashboard presents total medicine count, total available stock, low-stock medicines, batches expiring soon, and expired batches. Charts summarize stock by category and recent inbound/outbound movement. Medicine and batch pages provide search and filters; the batch page supports supplier and expiry-status filtering. Transaction pages record stock-in, stock-out, and adjustment activity. The buyer workflow includes browsing, product details, cart, checkout, order confirmation, order history, tracking, profile, and wishlist pages.

The technical scope includes a React 18 and Vite frontend, React Router navigation, a shared React Context state layer, custom CSS, Recharts visualizations, an Express backend, MySQL access through mysql2, and REST endpoints. The frontend API service includes local-storage persistence and backend request fallback. The supplied SQL file provides the MySQL schema. Deployment to a public production host, advanced forecasting, automatic supplier purchase orders, and integration with external pharmacy systems are not claimed as completed features.

1.4 Objectives of the Project

1. To develop a centralized web application for maintaining medicine inventory information.
2. To maintain medicine details such as generic name, category, manufacturer, strength, unit, price, and minimum stock threshold.
3. To track each medicine batch separately with supplier, manufacturing date, expiry date, and current quantity.
4. To calculate medicine-level stock from the quantities recorded in individual batches.
5. To identify expired batches, batches expiring soon, low-stock medicines, and out-of-stock medicines.
6. To maintain supplier contact details and associate suppliers with the batches they provide.
7. To record stock-in, stock-out, and adjustment transactions with a traceable audit history.
8. To validate stock issues so that expired stock and quantities exceeding the available balance cannot be issued.
9. To provide a dashboard with key indicators and charts for inventory monitoring.
10. To provide a technical foundation for future demand forecasting and replenishment planning.

1.5 Importance of the Application

The application is important because medicine stock must be monitored for both quantity and validity. An aggregate stock figure alone cannot indicate whether all units are safe to issue. Batch-wise tracking enables staff to inspect expiry dates and quantities separately. It also makes supplier traceability possible, since each batch is associated with a supplier.

The dashboard improves visibility by presenting low-stock and expiry-related conditions in one place. Search, filters, sorting, and pagination reduce the effort required to locate records. The transaction ledger helps explain changes in stock and supports reconciliation between system quantities and physical counts. These capabilities can reduce avoidable manual work and improve the organization of pharmacy inventory data.

1.6 Target Users / Audience

- Pharmacy administrators – maintain users, inventory records, and overall system settings.
- Inventory managers – monitor stock levels, create batches, record receipts, and perform adjustments.
- Pharmacists / dispensary staff – review medicine and batch details and record permitted stock issues.
- Supplier coordinators – maintain supplier information and review supplied batches.
- Buyers / customers – use the buyer-facing pages to browse available medicines and manage orders, where enabled.
- Project evaluators and developers – inspect the application modules, database design, API operations, and test workflows.

CHAPTER - 2
SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The project is a browser-based web application and does not require specialized hardware for normal use. A laptop or desktop with a modern multi-core processor, at least 8 GB RAM, and approximately 2 GB of free space for source code and development dependencies is suitable for development. An SSD is recommended for faster dependency installation and build operations. A stable network connection is useful when installing packages or accessing remote documentation.

For a local full-stack development setup, the computer should be capable of running the browser, Vite development server, Express API server, and MySQL Server at the same time. A display with a resolution of 1366 × 768 or higher is recommended for comfortable dashboard and table viewing. End users can access the application through a modern desktop or mobile browser.

Component	Recommended requirement	Purpose
Processor	Intel Core i5 / equivalent or better	Runs editor, browser, server, and database tools
Memory	8 GB RAM minimum; 16 GB preferred	Supports simultaneous development services
Storage	2 GB free project space; SSD preferred	Stores source, dependencies, database files, and builds
Display	1366 × 768 or higher	Displays dashboards and data tables clearly
Network	Stable broadband / LAN	Package installation and network access

2.2 Software Requirements

Software	Version / requirement	Usage
Operating System	Windows 10/11, Linux, or macOS	Development and execution environment
Node.js	18 or later recommended	Runs Vite tooling and Express server
npm	9 or later recommended	Dependency management and scripts
React	18.3.1	Component-based user interface
Vite	5.4.2	Development server and production build
Express	5.3.0	Backend REST API
MySQL Server	8.0 or later recommended	Relational data storage
MySQL Workbench	8.x	Database creation and SQL execution
Web Browser	Current Chrome, Edge, or Firefox	Application access and testing
IDE	Visual Studio Code or equivalent	Source editing and debugging

2.3 Development Tools and Frameworks

Visual Studio Code can be used to edit JSX, JavaScript, CSS, and SQL files. The terminal is used to install dependencies, start the frontend and backend processes, and run the production build. Vite provides the frontend development server and build command. React Router DOM manages page navigation. Lucide React supplies interface icons, and Recharts renders dashboard charts.

The backend uses Express to define API routes and middleware. The mysql2/promise package provides asynchronous access to MySQL. dotenv loads database configuration from the environment file, and cors enables requests from the configured local frontend origins. MySQL Workbench can execute the supplied MediStock_MySQL_Workbench_8.sql script and inspect tables and relationships.

Git and GitHub may be used for version control and team collaboration. Browser developer tools help inspect network requests, JavaScript errors, responsive layouts, and rendered components. The project includes simple test-flow scripts for buyer and fulfillment flows; manual functional checks should also be performed for inventory operations.

CHAPTER - 3
TECHNOLOGY STACK

3.1 Front-End

The frontend is developed with React 18. React components divide the application into reusable elements such as page headers, data tables, search bars, filters, modals, status badges, pagination controls, statistic cards, navigation, and notification messages. This component-based approach helps maintain a consistent interface across inventory pages.

Vite is used as the development and build tool. React Router DOM v6 handles client-side navigation between dashboard, medicine, batch, supplier, transaction, authentication, and buyer pages. Custom CSS files define the layout, design tokens, responsive behavior, forms, tables, and dashboard appearance. Lucide React provides icons, while Recharts is used for bar and area charts.

3.2 Back-End

The project archive contains an Express server in the server directory. The server loads environment settings using dotenv, parses JSON request bodies, enables CORS for the local frontend origins, and mounts route groups under /api/v1. The route groups cover authentication, medicines, batches, suppliers, transactions, orders, and health checks. The server listens on port 5000 by default.

The backend route handlers use parameterized SQL queries through a MySQL connection pool. The medicine endpoint returns medicine information together with aggregate stock values derived from batch rows. Batch endpoints retrieve batch, medicine, and supplier information and create stock-in records when a new batch is added. Supplier endpoints support supplier retrieval and maintenance. Transaction endpoints record stock movement and update batch quantities for relevant transaction types.

3.3 Database

MySQL is the relational database management system specified by the supplied SQL script. The database is named medistock_db and uses InnoDB with utf8mb4 character encoding. The schema contains core entities for stores, users, user addresses, suppliers, medicines, medicine batches, shopping carts, cart items, orders, and inventory transactions, along with supporting order and authentication structures.

The medicine_batches table is central to inventory tracking. It links to medicines and suppliers through foreign keys and stores batch number, manufacturing date, expiry date, initial quantity, current quantity, purchase price, and quarantine status. The stock_transactions table maintains a record of stock changes. Constraints and indexes support data integrity and common lookups such as expiry date, supplier, and quantity.

3.4 Version Control

Git provides local version history and supports branching, merging, and tracking modifications. GitHub can be used as a shared remote repository for the four-member team. The project can be divided into frontend modules, backend routes, database design, testing, and documentation responsibilities. Environment files containing passwords should not be committed to a public repository.

3.5 APIs and External Services

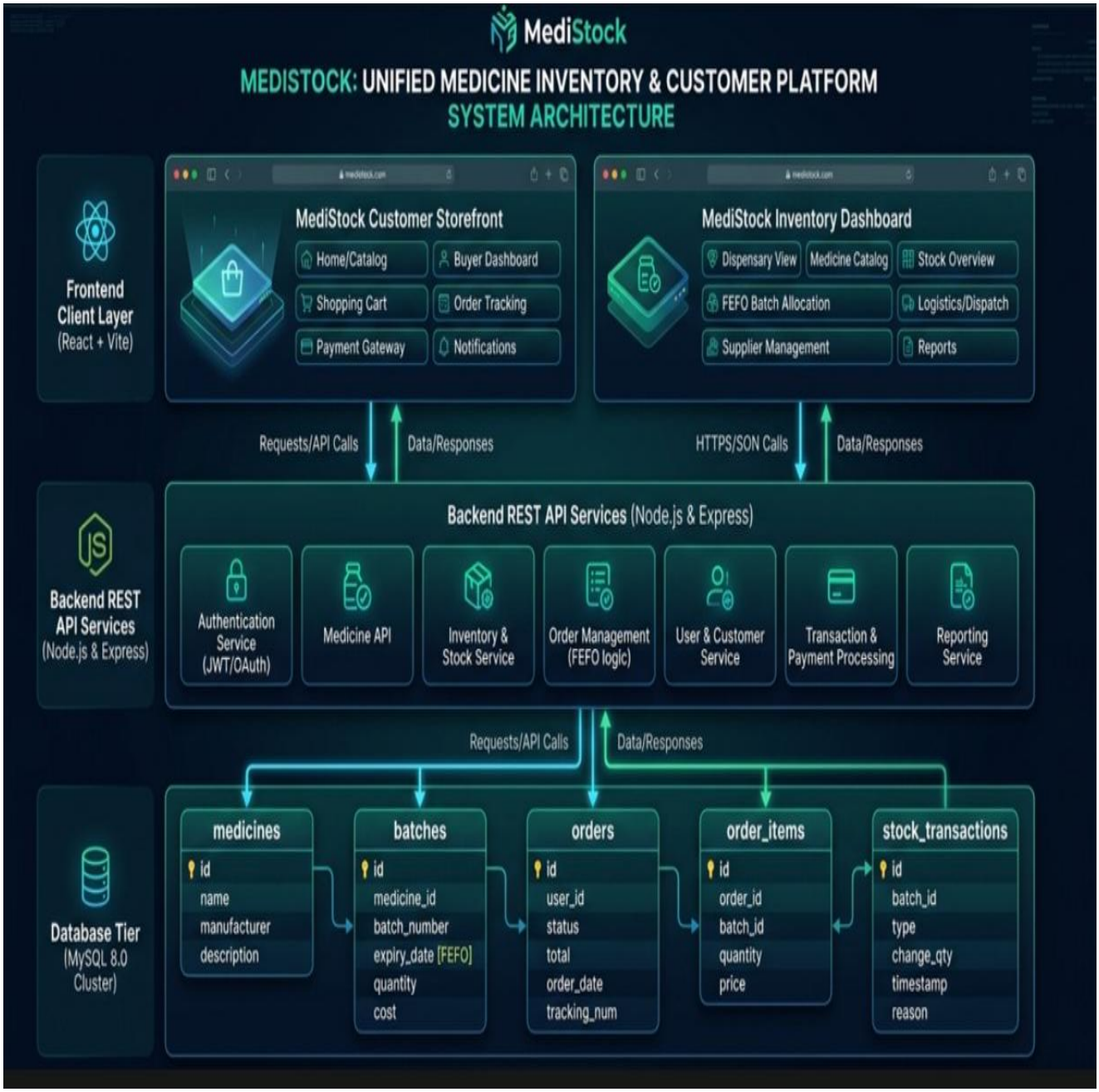
The backend exposes REST-style endpoints under /api/v1. The frontend service module calls the API base URL http://localhost:5000/api/v1 and contains fallback behavior that uses localStorage when backend requests

are unavailable. This permits the frontend to remain usable in a local demonstration environment. The SQL script is intended to be executed in MySQL Workbench before database-backed operations are tested.

Route group	Example endpoint	Purpose
Authentication	/api/v1/auth	User login and related access operations
Medicines	/api/v1/medicines	Read, create, update, and delete medicine records
Batches	/api/v1/batches	Manage batch quantities, supplier links, and expiry dates
Suppliers	/api/v1/suppliers	Maintain supplier records
Transactions	/api/v1/transactions	Record stock-in, stock-out, and adjustment activity
Orders	/api/v1/orders	Support buyer order workflows
Health	/api/v1/health	Check API and database status

CHAPTER - 4
SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram



CHAPTER - 5
DESIGN

5.1 Functional Design

The application is organized around the lifecycle of medicine inventory. A medicine master record describes the product, while one or more batch records describe the physical lots received. Each batch is linked to one supplier and one medicine. The quantity available for a medicine is derived from its batch quantities, with expired or quarantined stock excluded from usable stock calculations in the frontend inventory context.

The main functional flow begins when an authorized user creates a medicine record. A batch can then be registered with supplier, dates, quantity, and purchase price. Stock receipts are recorded as stock-in activity. When stock is issued, the user selects a medicine and a valid batch, enters a quantity, and provides a reason. An adjustment records the verified physical quantity and supports reconciliation.

5.2 ER Diagram / Database Schema

The following entities and relationships are represented in the supplied SQL schema. The exact database diagram may be added by the project team in the reserved design space if required by the evaluation format.

Entity / Table	Important attributes	Relationship / purpose
stores	id, store_code, name, license_number, contact details	Store or pharmacy identity; may be associated with users
users	id, store_id, role, full_name, email, password_hash, status	User accounts with role-based classification
suppliers	id, name, contact_person, email, phone, address, status	Supplier master records
medicines	id, name, generic_name, category, manufacturer, strength, unit, min_stock_threshold, selling_price	Medicine catalogue
medicine_batches	id, medicine_id, batch_number, supplier_id, mfg_date, expiry_date, quantity, purchase_price, quarantined	Many batches belong to one medicine; each batch refers to a supplier
stock_transactions	id, medicine_id, batch_id, type, quantity, reason, performed_by, transaction_date	Movement history linked to medicine and optionally a batch
user_addresses	id, user_id, address_label, recipient_name, street, city, postal_code	Buyer delivery addresses
shopping_carts / cart_items	cart id, user id, medicine id, quantity	Buyer cart and its medicine lines
orders / order items	order id, buyer, order status, payment/delivery data, medicine lines	Buyer order and fulfillment workflow

5.3 Data Validation and Integrity

- Medicine identifiers and required descriptive fields must be supplied when creating a medicine.
- Batch records require medicine, batch number, supplier, manufacturing date, expiry date, and quantity.
- Manufacturing date must not be later than the expiry date.
- Batch quantity and purchase price must be non-negative; the SQL schema defines checks for these values.
- Medicine batch numbers are unique within a medicine according to the database constraint.
- Foreign keys prevent deleting medicines or suppliers while dependent batch records remain.

- Stock-out validation checks the available quantity and prevents issuing expired stock in the frontend workflow.
- Search and filtering are applied to medicine, batch, and supplier attributes to help users locate records.

CHAPTER - 6
IMPLEMENTATION

6.1 Module-Wise Implementation

6.1.1 Medicine Management

The Medicines page retrieves medicine records from the shared inventory context and displays them in a searchable, sortable, paginated table. Search supports medicine name, generic name, medicine ID, and manufacturer. Category and stock-status filters narrow the displayed records. Users can open a details modal, add a medicine, edit an existing record, or delete a record when dependency rules permit. The medicine form captures product details such as category, strength, unit, manufacturer, threshold, selling price, and prescription requirement.

6.1.2 Batch and Expiry Management

The Batches page manages individual medicine lots. It displays batch identifiers, batch numbers, medicine, supplier, manufacturing date, expiry date, quantity, and related status. Users can search by batch number, medicine name, or batch ID, and filter by supplier and expiry status. The page supports sorting and pagination. The expiry utility classifies batches as expired, expiring soon, or valid based on the current date. A batch details modal presents the selected lot information.

6.1.3 Supplier Management

The Suppliers page maintains company name, contact person, email, phone number, address, status, notes, and the number of associated batches. Users can search by supplier name, contact person, email, phone, or supplier ID. Supplier details include the batches associated with that supplier. Deletion is guarded when inventory batches depend on the supplier, protecting referential integrity.

6.1.4 Stock Transactions

The Transactions page records stock movement using transaction types such as STOCK_IN, STOCK_OUT, and ADJUSTMENT. A transaction contains medicine, batch, quantity, reason, operator, and timestamp. Stock-in increases inventory, stock-out decreases the selected batch quantity, and adjustment records a physical count. The user interface includes a review/confirmation step and a transaction details view showing the quantity movement and balance context.

6.1.5 Dashboard and Analytics

The dashboard combines inventory statistics and quick actions. The project calculates total medicines, total available stock, low-stock medicine count, expiring-soon batch count, and expired batch count from the current inventory data. A category chart visualizes stock distribution, while a seven-day movement chart summarizes inbound and outbound transactions. The dashboard also provides preview lists for low-stock medicines, expiry alerts, and recent transactions.

6.1.6 Buyer and Order Workflow

The supplied source archive includes buyer pages for browsing the medicine catalogue, viewing product details, managing a cart, checking out, confirming orders, viewing order history, tracking an order, managing a profile, and maintaining a wishlist. This area is separate from the inventory-management workflow and uses buyer-specific context and components. The SQL schema includes users, addresses, carts, cart items, and order-related data structures to support this workflow.

6.2 Front-End Logic and State Management

InventoryContext is the shared state provider for medicines, batches, suppliers, transactions, orders, loading state, and toast notifications. It loads data through service methods and derives enriched records. Batch records are enriched with medicine and supplier names and expiry status. Medicine records are enriched with stock totals and usable-stock values calculated from related batches. This avoids maintaining a separate manually edited aggregate quantity for each medicine.

React useState manages local page state such as search text, selected filters, sorting, pagination, and modal visibility. useMemo is used for derived filtered and sorted lists. Reusable components provide consistent tables, forms, dialogs, status badges, and notifications. The service layer persists data in browser localStorage and attempts backend API requests, allowing the application to continue in a demo mode when the backend is not reachable.

6.3 API Integration

The Express backend mounts route modules for authentication, medicines, batches, suppliers, transactions, orders, and health checks. The MySQL connection module creates a connection pool using DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, and DB_NAME environment values. Parameterized queries are used in route handlers. The frontend service layer centralizes data operations so that UI pages do not directly construct SQL queries.

The medicine GET endpoint joins medicines with batches and calculates totalStock and usableStock. The batch GET endpoint joins batch data with medicine and supplier details and orders records by expiry date. The supplier GET endpoint returns supplier information and the count of associated batches. The transaction GET endpoint joins transaction data with medicine and batch information. The API server uses port 5000 by default and the frontend development server uses Vite.

6.4 Authentication and Authorization

The application contains an authentication page, an authentication context, and a protected-route component. The database schema defines user roles including BUYER, INVENTORY_MANAGER, PHARMACIST, and ADMIN, along with account status and a password_hash field. The buyer and staff experiences are routed separately. The database comments specify that passwords should be stored as secure BCrypt or Argon2 hashes, never as plaintext. Production deployment should verify password hashing, session/token handling, role checks on every protected API route, and secure secret management before real patient or pharmacy data is used.

CHAPTER - 7

FEATURES

7.1 List of Main Features

- Medicine catalogue with create, read, update, delete, details, search, category filter, and stock-status filter.
- Batch-level stock management with supplier association and manufacturing/expiry date tracking.
- Expiry status classification for expired, expiring-soon, and valid batches.
- Supplier directory with contact details, associated batch count, search, and dependency-aware deletion.
- Stock transaction ledger for stock-in, stock-out, and physical stock adjustment.
- Dashboard statistics, category stock chart, recent stock movement chart, and critical inventory alerts.
- Buyer-side medicine browsing, product details, cart, checkout, order confirmation, order history, tracking, profile, and wishlist pages.
- Local browser persistence and REST API / MySQL integration components.

7.2 How Each Feature Works

Feature	User perspective	Technical behavior
Medicine catalogue	Search or maintain medicine products	React page uses InventoryContext; service methods persist or request API data
Batch management	Register and inspect medicine lots	Batch records link medicine and supplier; expiry status is date-derived
Expiry tracking	Identify expired and soon-to-expire stock	Date utility compares expiry date with current date and assigns status
Supplier directory	Maintain vendor contacts and supplied batches	Supplier records are joined to batches; deletion checks dependencies
Stock-in	Record newly received quantity	Creates a batch and logs a STOCK_IN movement
Stock-out	Issue a quantity from a selected batch	Validates quantity and expiry before reducing batch stock
Adjustment	Correct system quantity after physical count	Records an adjustment and updates the batch quantity
Dashboard	View inventory condition at a glance	Aggregates derived stock and transaction data; renders Recharts charts
Buyer order flow	Browse products and place an order	Buyer pages and order API route manage cart and order lifecycle

7.3 Demand Forecasting Status

Demand forecasting is included in the project requirement statement as a desired capability. The inspected source code contains dashboard analytics for historical stock movement and inventory summaries, but no dedicated forecasting algorithm, trained model, forecast endpoint, or future-demand output was identified. Therefore, this documentation does not describe demand forecasting as an already implemented function. It is proposed as a future module that can estimate future medicine demand using historical issue quantities, order history, seasonality, lead time, and stock-out patterns.

A future forecasting module may calculate a moving average or weighted moving average for each medicine. A more advanced version may compare seasonal patterns and apply exponential smoothing or a time-series model. The forecast should be displayed with its time horizon, assumptions, confidence range, and

recommended reorder quantity. Forecasts should support—not automatically replace—review by an inventory manager.

CHAPTER - 8

TESTING

8.1 API Testing

The Express API can be tested using Postman or an equivalent REST client. Start the backend server and verify its root status endpoint and health route. Then test GET, POST, PUT, and DELETE operations for medicines and suppliers; GET and POST operations for batches and transactions; and the order routes used by the buyer flow. Verify both successful responses and validation/error responses.

Test area	Example test	Expected result
API status	GET /	Returns API online status and database connection information
Medicine list	GET /api/v1/medicines	Returns medicine records with calculated stock totals
Batch list	GET /api/v1/batches	Returns batches with medicine, supplier, and expiry fields
Supplier list	GET /api/v1/suppliers	Returns supplier records and associated batch counts
Transaction list	GET /api/v1/transactions	Returns chronological stock movement records
Invalid batch	POST batch with missing required values	Returns a client error describing required fields
Database unavailable	Stop MySQL and request a database-backed route	Returns an API error; frontend service may use local persistence

8.2 Integration Testing

Integration testing verifies that page components, the shared context, service layer, Express routes, and MySQL schema work together. A medicine should be created before a batch referencing it. A supplier should exist before a batch is registered. After adding a batch, the medicine-level stock summary should reflect the batch quantity. A stock-out should reduce the batch quantity and be visible in the transaction ledger. An adjustment should update the quantity to the entered physical count.

8.3 Tools Used for Testing

- Browser testing with Chrome or Edge developer tools.
- Postman or a similar API client for REST endpoint verification.
- MySQL Workbench for checking tables, constraints, and inserted records.
- Vite build command (npm run build) to check production compilation.
- Included buyer and fulfillment test-flow scripts as starting points for workflow checks.

8.4 Test Cases and Results

TC ID	Scenario	Expected behavior / validation
TC-01	Add a medicine with valid details	Medicine appears in catalogue and can be searched
TC-02	Search using generic name	Matching medicine records are displayed
TC-03	Create a batch with valid dates	Batch appears with supplier and medicine details

TC-04	Enter manufacturing date after expiry date	Form validation rejects the invalid date range
TC-05	View an expired batch	Batch receives EXPIRED status and is not usable stock
TC-06	View a batch expiring within 30 days	Batch is classified as expiring soon
TC-07	Stock-out greater than available quantity	Operation is rejected
TC-08	Stock-out from expired batch	Operation is rejected
TC-09	Record a physical quantity adjustment	Batch quantity is updated and adjustment is logged
TC-10	Delete supplier linked to batches	Deletion is blocked or rejected by dependency constraints
TC-11	Reload the browser after a local change	Persisted localStorage data remains available
TC-12	Run production build	Vite completes the frontend build without compilation errors

Note: The table describes the expected test cases derived from the supplied implementation and its business rules. It is not a claim that every test was executed in this documentation session. The project team should record actual execution date, input data, observed result, and pass/fail status during final validation.

CHAPTER - 9
DEPLOYMENT

9.1 Steps to Deploy / Run Locally

11. Extract the MEDICINE_INV project folder from the supplied ZIP archive.
12. Install Node.js and npm on the development computer.
13. Open a terminal in the MEDICINE_INV project directory.
14. Run npm install to install the packages listed in package.json.
15. For database-backed operation, start MySQL Server and open MySQL Workbench.
16. Execute MediStock_MySQL_Workbench_8.sql to create the medistock_db database and its tables.
17. Configure the database connection values in the project environment file.
18. Start the backend in one terminal using npm run server.
19. Start the frontend in another terminal using npm run dev.
20. Open the Vite local URL shown in the terminal, normally http://localhost:5173.

9.2 Environment Configuration

The backend reads database settings from environment variables. The project defaults are DB_HOST=localhost, DB_PORT=3306, DB_USER=root, DB_NAME=medistock_db, and an empty password if no value is provided. These defaults should be adjusted to match the local MySQL installation. The server uses PORT=5000 unless another port is supplied. Real credentials must be stored privately and must not be published in source control.

Variable	Example / default	Description
DB_HOST	localhost	MySQL host
DB_PORT	3306	MySQL port; project log may mention a custom local port
DB_USER	root	Database user
DB_PASSWORD	(configure locally)	Database password
DB_NAME	medistock_db	Database name
PORT	5000	Express server port

9.3 Hosting of Front End and Back End

The supplied archive includes a production frontend build directory, but no verified public hosting configuration or production URL. For a production deployment, the frontend can be hosted on a static web server or suitable cloud hosting service, while the Express API can run on a Node.js application host. MySQL should run on a managed or secured database server. The frontend API base URL, CORS origin list, HTTPS configuration, environment secrets, backups, and role-based access controls must be updated for the production environment. Localhost settings are intended for development and should not be used unchanged in production.

CHAPTER - 10
CHALLENGES AND LIMITATIONS

10.1 Issues Faced During Development

Inventory records are interdependent. A batch cannot be meaningful without a medicine and supplier, and deleting a medicine or supplier that is referenced by a batch may damage data consistency. Another challenge is calculating a correct medicine-level quantity when the same medicine is stored across multiple batches. The system addresses this by deriving aggregate stock from related batch records.

Expiry calculations require date-aware logic and consistent handling of date values. The project distinguishes expired, expiring-soon, and valid batches. Stock transactions also require validation because a stock issue must not exceed the available quantity or use an expired batch. The interface uses confirmation and detail views to make transaction actions clearer.

10.2 Solutions Applied

- Use a shared InventoryContext to maintain a common view of medicines, batches, suppliers, and transactions.
- Calculate medicine stock from batch-level quantities rather than maintaining a separate manually edited total.
- Use a date utility to classify expiry status and calculate days until expiry.
- Use reusable forms, modals, tables, filters, sorting, and pagination to keep pages consistent.
- Use database foreign keys and restrictive delete behavior to protect dependent records.
- Use a service layer with localStorage persistence and backend request support to separate data access from page components.
- Use parameterized MySQL queries in backend route handlers.

10.3 Current Limitations or Known Gaps

The frontend API service includes localStorage fallback behavior, so a local demonstration may operate without a live MySQL connection. This is useful for resilience but means that users must confirm whether a particular run is using browser-local data or the database backend. The backend and frontend must be configured consistently before database persistence can be treated as authoritative.

A dedicated medicine demand forecasting algorithm is not present in the inspected source. The dashboard visualizes recent stock movement but does not produce a future demand estimate. Automated reorder suggestions, supplier purchase-order generation, notifications through email/SMS, barcode scanning, and external pharmacy integrations are not established as completed features. Production security, deployment, backups, and operational monitoring require additional validation before real-world use.

The supplied project includes a .env file. Database credentials and other secrets should be reviewed and replaced with private deployment values. Any exposed credentials should be rotated. The project should not be deployed with demo accounts, weak passwords, permissive CORS settings, or unverified authorization checks.

CHAPTER - 11
FUTURE ENHANCEMENTS

11.1 Planned Features

- Medicine demand forecasting based on historical stock-out, dispensing, and buyer order records.
- Automatic reorder-level calculation using average demand, supplier lead time, and safety stock.
- Low-stock notifications and expiry alerts through email, SMS, or in-app notifications.
- Purchase-order creation and supplier delivery tracking.
- Barcode or QR-code scanning for faster medicine and batch identification.
- First-Expiry-First-Out (FEFO) issue recommendations for selecting batches.
- Advanced reports for stock valuation, wastage, expiry loss, supplier performance, and monthly consumption.
- Role-specific permissions, audit logs, secure password hashing, and stronger session/token management.
- Cloud deployment, automated backups, monitoring, and disaster recovery.
- Integration with billing, electronic prescriptions, and pharmacy management systems.

11.2 Demand Forecasting and Optimization

Demand forecasting can be introduced as a separate analytics service. The input dataset may include daily or weekly stock-out quantities, fulfilled buyer orders, manual dispensing records, stock availability, and seasonal indicators. Data should be cleaned and grouped by medicine identifier. Missing dates can be represented as zero demand only when the business meaning is confirmed; otherwise missing records must be distinguished from zero sales.

A simple first version can use a moving average over recent periods. A weighted moving average can give greater importance to recent demand. Exponential smoothing can account for changing demand, while more advanced time-series models may be evaluated when sufficient historical data is available. Forecast accuracy can be measured using MAE or MAPE on a held-out time period.

A reorder recommendation can combine predicted demand during supplier lead time with safety stock. A general formula is: Reorder Point = Forecast Demand During Lead Time + Safety Stock. The recommended purchase quantity should also consider current usable stock, pending orders, quarantined stock, and quantities already on order. Any recommendation should be reviewed by an authorized inventory manager before a purchase order is placed.

CHAPTER - 12
CONCLUSION

12.1 Summary of the Project

The Medicine Inventory Management project provides a structured web application for maintaining medicine catalogue data, supplier records, batch-level quantities, expiry information, and stock transactions. It replaces a fragmented record-keeping approach with dedicated modules and a dashboard that presents important inventory indicators. The project also contains a buyer-facing medicine ordering workflow.

12.2 What Was Achieved

The supplied implementation includes React-based pages for medicines, batches, suppliers, transactions, dashboard, authentication, and buyer activities. It includes reusable interface components, shared inventory state, search and filtering, sorting, pagination, modals, notifications, expiry classification, stock calculations, and transaction validation. An Express API and MySQL schema provide a backend persistence path. The SQL script defines relationships and integrity constraints for the core inventory entities and buyer workflow.

The main achievement is the organization of inventory around medicine batches rather than a single medicine-level quantity. This makes expiry monitoring and supplier traceability possible. The transaction ledger provides a history of stock movement, and the dashboard provides a convenient overview of stock condition. Demand forecasting remains a planned extension and should be implemented and tested as a separate module.

12.3 Skills Learned During Development

- React component development, hooks, context-based state management, and routing.
- Frontend form design, validation, reusable components, and responsive CSS.
- REST API design and integration using Express.
- Relational database design, SQL queries, foreign keys, indexes, and constraints in MySQL.
- Inventory concepts such as batch tracking, expiry monitoring, stock thresholds, and stock reconciliation.
- Testing of user workflows, API endpoints, database operations, and production builds.
- Team collaboration, modular project organization, documentation, and technical presentation.

CHAPTER - 13

REFERENCES

- React Documentation – https://react.dev/
- Vite Documentation – https://vite.dev/guide/
- React Router Documentation – https://reactrouter.com/
- Express Documentation – https://expressjs.com/
- MySQL 8.0 Reference Manual – https://dev.mysql.com/doc/
- MySQL2 Node.js Driver – https://github.com/sidorares/node-mysql2
- Recharts Documentation – https://recharts.org/
- Lucide Icons – https://lucide.dev/
- Node.js Documentation – https://nodejs.org/docs/
- MDN Web Docs – https://developer.mozilla.org/
- Project source archive: MEDICINE_INV_phase5.zip (frontend, backend, SQL schema, and README).
- Project reference template: DBSE_Project_Documentation.docx.

CHAPTER - 14

APPENDICES

Appendix A – Installation and Setup Guide

1. Extract the project ZIP file. 2. Open a terminal in the MEDICINE_INV folder. 3. Run npm install. 4. Start MySQL and execute the supplied SQL script if database-backed operation is required. 5. Configure DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, and DB_NAME in the environment file. 6. Run npm run server in one terminal. 7. Run npm run dev in a second terminal. 8. Open the local Vite URL in a browser.

Appendix B – User Guide

21. Sign in through the application entry page using the configured demo or authorized account.
22. Open the dashboard to review stock indicators and expiry alerts.
23. Use Medicines to create or update medicine master records.
24. Use Suppliers to maintain supplier contact information.
25. Use Batches to register received lots with manufacturing and expiry dates.
26. Use Transactions to record stock-in, stock-out, or physical-count adjustments.
27. Search, filter, sort, and paginate tables to locate records.
28. Review the transaction ledger after a stock operation to verify the recorded movement.
29. Use buyer pages only when the buyer-facing workflow is enabled and configured.

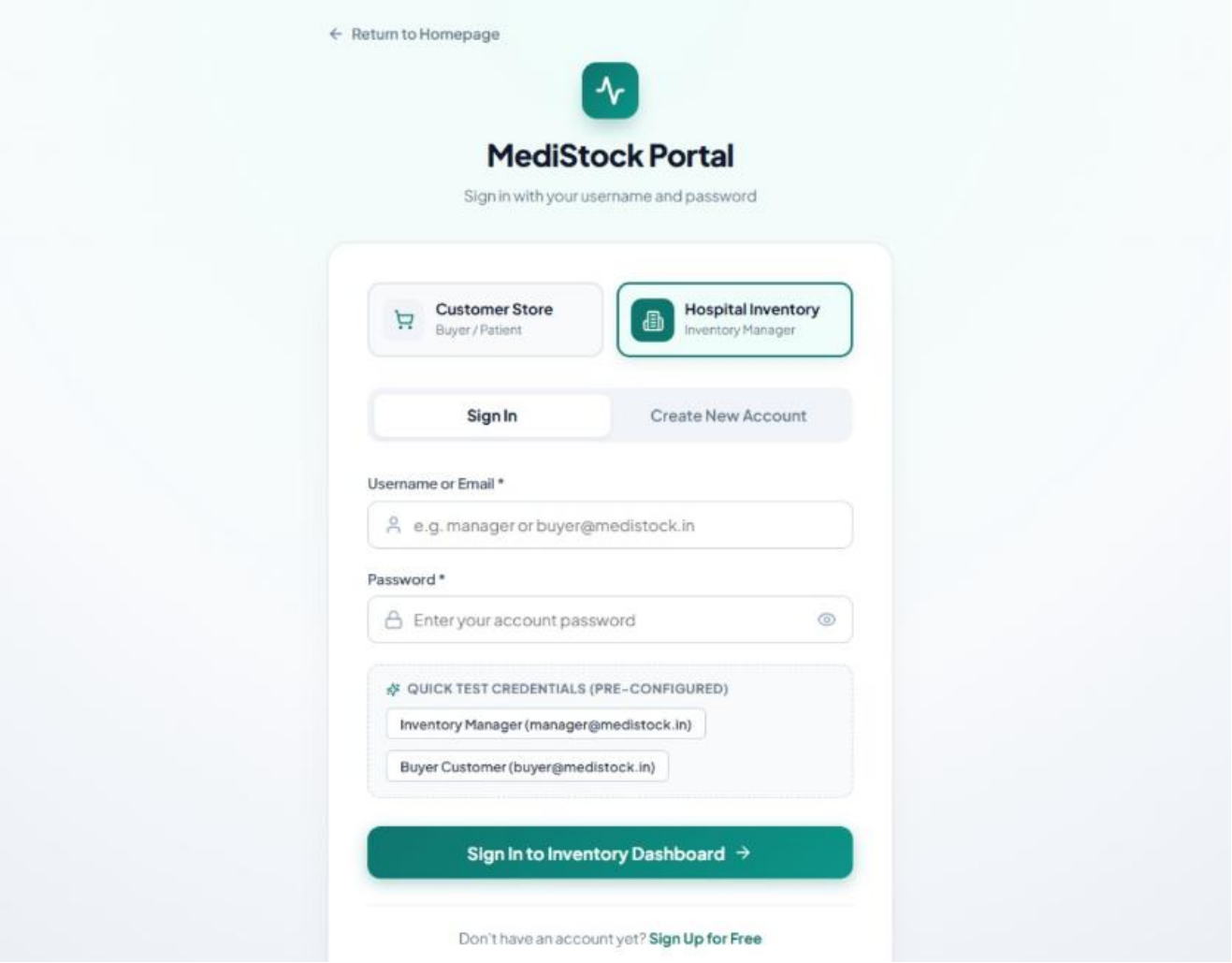
Appendix C – Sample API Route Reference

Method	Endpoint	Purpose
GET	/api/v1/medicines	List medicines and derived stock values
POST	/api/v1/medicines	Create a medicine
GET	/api/v1/batches	List batches and expiry information
POST	/api/v1/batches	Create a batch and log receipt
GET	/api/v1/suppliers	List suppliers
POST / PUT / DELETE	/api/v1/suppliers	Maintain supplier records
GET / POST	/api/v1/transactions	Read and record stock movements
GET	/api/v1/health	Check API/database health

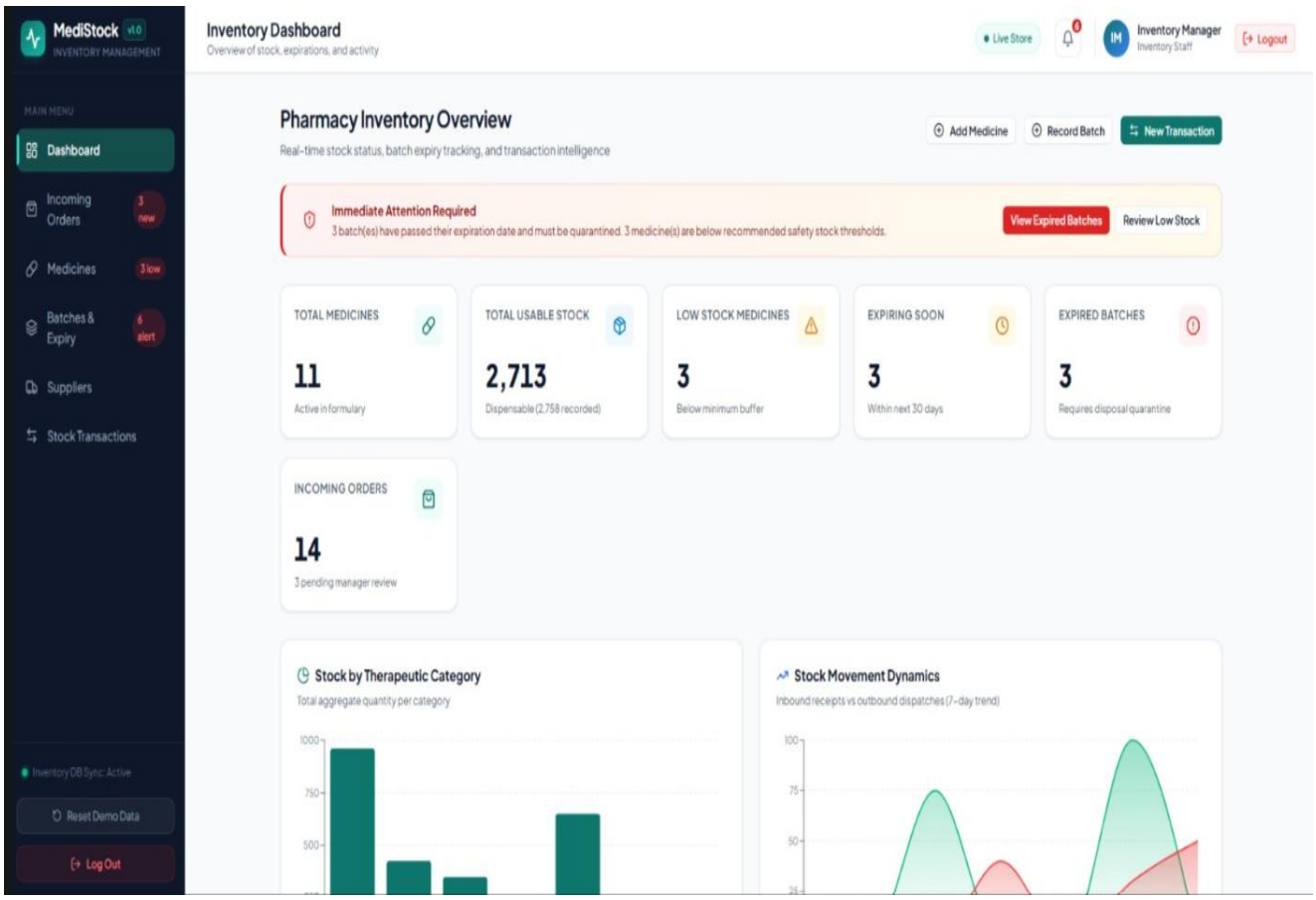
Appendix D – Website Screenshots

The following pages are intentionally left blank for the project team to insert actual screenshots captured from the running website. Add genuine application screenshots only; do not use illustrative or recreated images as evidence of execution.

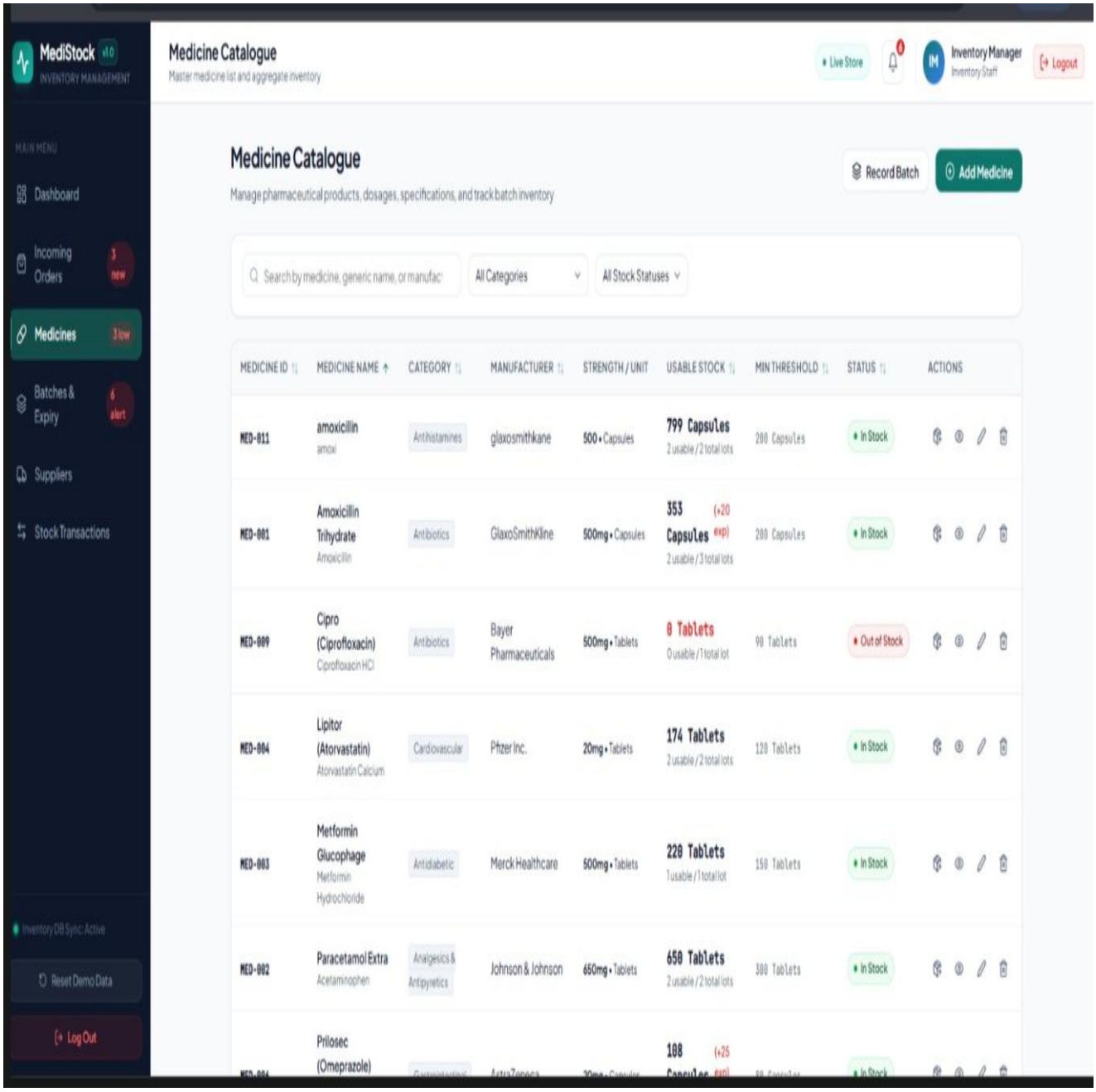
D.1 Login / Authentication Page



D.2 Inventory Dashboard

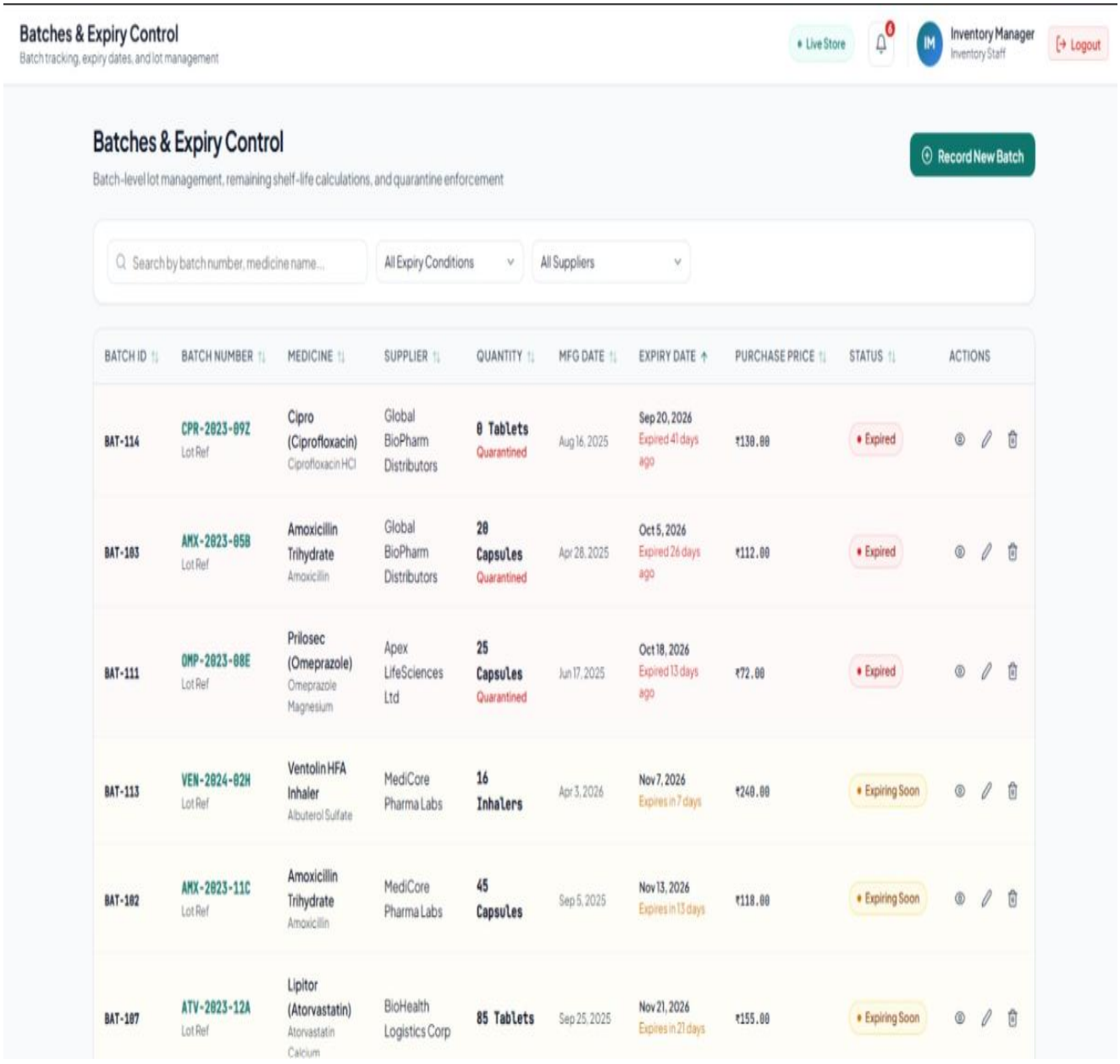


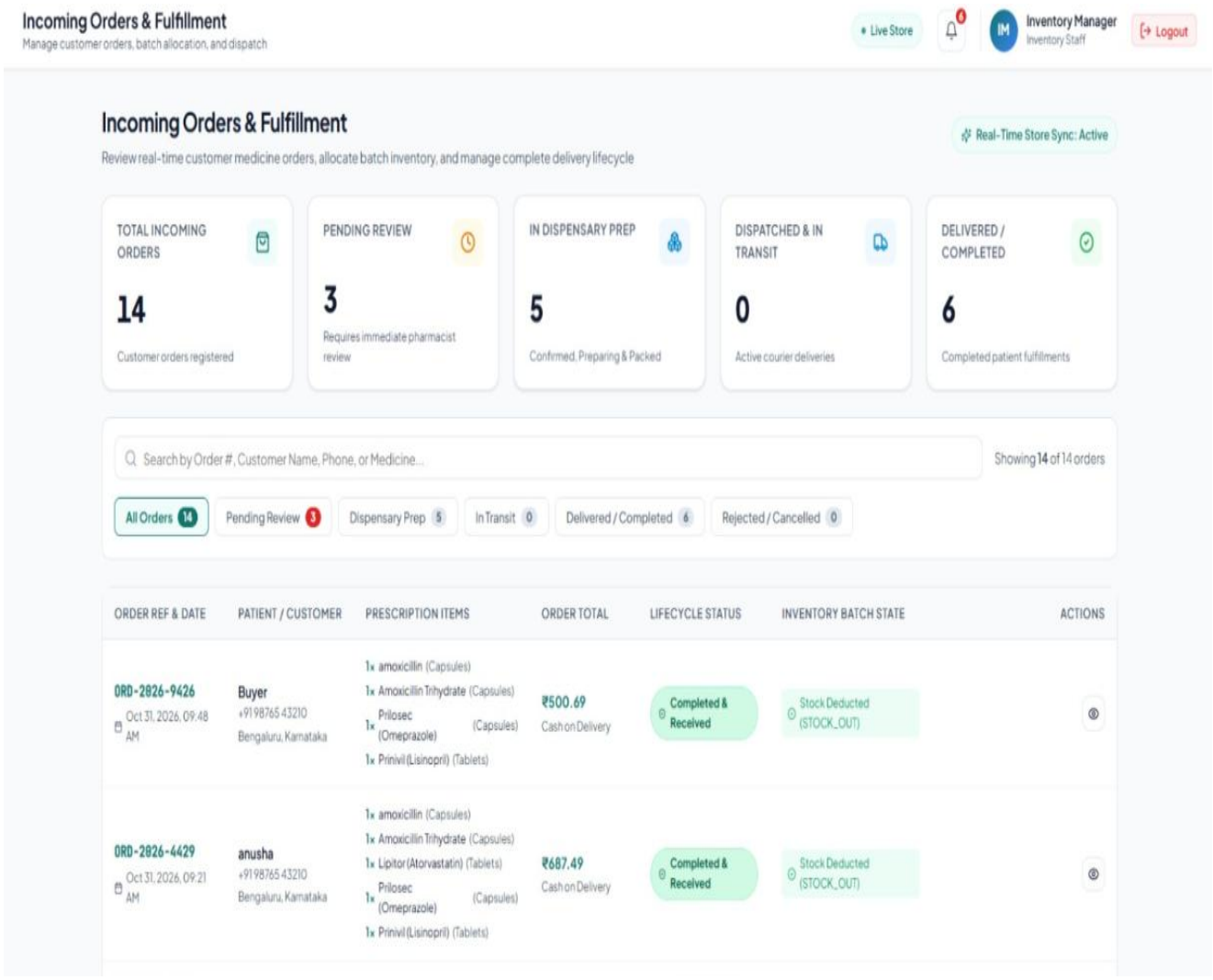
D.3 Medicine Catalogue

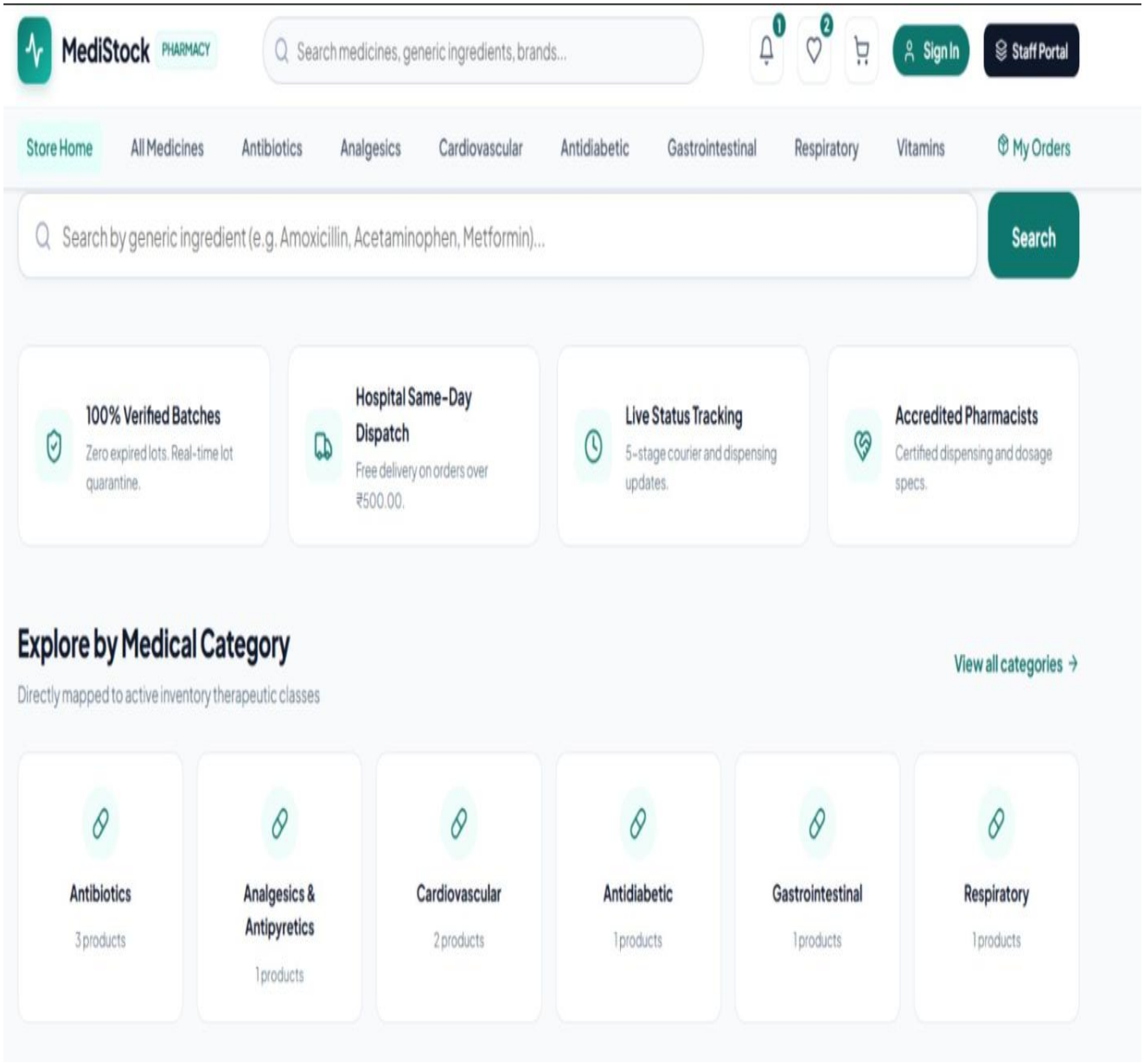


\

D.4 Batch and Expiry Management




Incoming Orders & Fulfillment
Manage customer orders, batch allocation, and dispatch
Live Store
Inventory Manager
Inventory Staff
Logout
Incoming Orders & Fulfillment
Review real-time customer medicine orders, allocate batch inventory, and manage complete delivery lifecycle
Real-Time Store Sync: Active
TOTAL INCOMING ORDERS
14
Customer orders registered
PENDING REVIEW
3
Requires immediate pharmacist review
IN DISPENSARY PREP
5
Confirmed, Preparing & Packed
DISPATCHED & IN TRANSIT
0
Active courier deliveries
DELIVERED / COMPLETED
6
Completed patient fulfillments
Search by Order #, Customer Name, Phone, or Medicine...
Showing 14 of 14 orders
All Orders 14
Pending Review 3
Dispensary Prep 5
In Transit 0
Delivered / Completed 6
Rejected / Cancelled 0
ORDER REF & DATE
PATIENT / CUSTOMER
PRESCRIPTION ITEMS
ORDER TOTAL
LIFECYCLE STATUS
INVENTORY BATCH STATE
ACTIONS
ORD-2026-9426
Oct 31, 2026, 09:48 AM
Buyer
+919876543210
Bengaluru, Karnataka
1x amoxicillin (Capsules)
1x Amoxicillin Trihydrate (Capsules)
1x Prilosec (Omeprazole) (Capsules)
1x Prinivil (Lisinopril) (Tablets)
₹500.69
Cash on Delivery
Completed & Received
Stock Deducted (STOCK_OUT)
ORD-2026-4429
Oct 31, 2026, 09:21 AM
anusha
+919876543210
Bengaluru, Karnataka
1x amoxicillin (Capsules)
1x Amoxicillin Trihydrate (Capsules)
1x Lipitor (Atorvastatin) (Tablets)
1x Prilosec (Omeprazole) (Capsules)
1x Prinivil (Lisinopril) (Tablets)
₹687.49
Cash on Delivery
Completed & Received
Stock Deducted (STOCK_OUT)


MediStock
PHARMACY
Search medicines, generic ingredients, brands...
Sign In
Staff Portal
Store Home
All Medicines
Antibiotics
Analgesics
Cardiovascular
Antidiabetic
Gastrointestinal
Respiratory
Vitamins
My Orders
Search by generic ingredient (e.g. Amoxicillin, Acetaminophen, Metformin)...
Search
100% Verified Batches
Zero expired lots. Real-time lot quarantine.
Hospital Same-Day Dispatch
Free delivery on orders over ₹500.00.
Live Status Tracking
5-stage courier and dispensing updates.
Accredited Pharmacists
Certified dispensing and dosage specs.
Explore by Medical Category
Directly mapped to active inventory therapeutic classes
View all categories →
Antibiotics
3 products
Analgesics & Antipyretics
1 products
Cardiovascular
2 products
Antidiabetic
1 products
Gastrointestinal
1 products
Respiratory
1 products